
Nexidia Real-Time Grid JMS TLS Guide

Nexidia Real-Time Grid JMS TLS Guide

Version 12.5.1

Table of Contents

Copyright

Overview

Use a Trusted Certificate

ActiveMQ Artemis JMS/TLS

ActiveMQ JMS/TLS

 Python Client

 Python Client Trusted Root CA Import

 Python Client Certificate

 Use OpenSSL to Generate the Client Key and Certificate

 Python Example Code

 Run the Python Example

 Java Client

 Java Client Certificate

 Use OpenSSL to Generate the Client Key and Certificate

 Java Client Trusted Root CA Import

 Java Example Code

 Java Example Client Main Class

 Java Example Client Transcript Consumer

 Java Example Client Command File

Copyright

PROPRIETARY AND CONFIDENTIAL INFORMATION

Information herein is proprietary information and trade secrets of NICE Ltd. and/or its affiliated companies (Affiliates). This document and the information herein is the exclusive property of NICE and its Affiliates and shall not be disclosed, in whole or in part, to any third party or utilized for any purpose other than the express purpose for which it has been provided.

https://wem-dochub.mindtouch.us/Nexidia_Analytics_and_Data_Exchange_Framework/NXQC_12.5/Grid_Documentati...

Updated: Thu, 30 Apr 2026 18:12:29 GMT

IMPORTANT NOTICE

Subject always to any existing terms and conditions agreed between you and NICE or any Affiliate with respect to the products which are the subject matter of this document, neither NICE nor any of its Affiliates shall bear any responsibility or liability to a client or to any person or entity with respect to liability, loss or damage caused or alleged to be caused directly or indirectly by any product supplied or any reliance placed on the content of this document. This includes, but is not limited to, any interruption of service, loss of business or anticipatory profits or consequential damage resulting from the use or operation of any products supplied or the content of this document. Information in this document is subject to change without notice and does not represent a commitment on the part of NICE or any Affiliate.

All information included in this document, such as text, graphics, photos, logos and images, is the exclusive property of NICE or in Affiliate and is protected by United States and international copyright laws. Permission is granted to use, view and photocopy (or print) materials from this document only in connection with the products to which this document relates and subject to the terms of license applicable to such products. Any other use, copying, distribution, retransmission or modification of the information in this document without the express prior written permission of NICE or an Affiliate is strictly prohibited. In the event of any permitted copying, redistribution or publication of copyrighted material, no changes in, or deletion of, author attribution, trademark legend or copyright notice shall be made.

Products supplied may be protected by one or more of the US patents listed at www.nice.com/Patents. For the full list of trademarks of NICE and its Affiliates, visit www.nice.com/Nice-Trademarks. All other marks used are the property of their respective proprietors.

All contents of this document are: Copyright © 2025 NICE Ltd. All rights reserved.

For assistance, contact your local supplier or nearest NICE Customer Service Center:

EMEA Region (Europe, Middle East, Africa) Tel: +972-9-775-3800 Fax: +972-9-775-3000 email: support@nice.com	APAC Region (Asia/Pacific) Tel: +852-8338-9818 Fax: +852-2802-1800 email: support.apac@nice.com	North America Tel: 1-800-663-5601 Fax: +201-356-2197 email: na_sales@nice.com	International Headquarters- Israel Tel: +972-9-775-3100 Fax: +972-9-775-3070 email: info@nice.com
The Americas Region (North, Central, South America) Tel: 1-800-6423-611 Fax: +720-264-4012 email: support.americas@nice.com	Israel Tel: 09-775-3333 Fax: 09-775-3000 email: support@nice.com	France Tel: +33-(0)1-41-38-5000 Fax: +33-(0)1-41-38-5001	Hong-Kong Tel: +852-2598-3838 Fax: +852-2802-1800
United Kingdom Tel: +44-8707-22-4000 Fax: +44-8707-22-4500	Germany Tel: +49-(0)-69-97177-0 Fax: +49-(0)-69-97177-200		

NICE invites you to join the NICE User Group (NUG).

All queries, comments, and suggestions are welcome! Please email: nicebooks@nice.com.

For more information about NICE, visit www.nice.com

Overview

This document describes how to configure a JMS client to use Transport Level Security (TLS) and the appropriate certificates to connect to the Real-Time Grid (RTG) Message Broker (MB) and to subscribe to receive Java Message Service (JMS) messages secured by TLS.

Instructions are provided for the use of TLS certificates with clients developed using Python or Java. The same principles and tools apply to clients developed using other programming languages and platforms.

Use a Trusted Certificate

For production environments, a certificate should be acquired from a trusted source such as Digicert, Thawte, Verisign, etc. These sources are known as Trusted Root Authorities. Certificates signed by a certificate that traces to one of these sources are recognized in the Certificate Authority (CA) list for Java, and may have current CA certificates in the Windows Certificate Store.

For hosted or hybrid RTG environments, NICE-Nexidia may provide a certificate signed by a Trusted Root CA. For on-premise environments the customer may purchase their own certificate signed by a Trusted Root CA.

ActiveMQ Artemis JMS/TLS

This section and subsections describe how clients can access the RTG MB when the MB is using ActiveMQ Artemis as the JMS messaging service and the JMS messages are secured by TLS.

- Add key store and trust store to Artemis MQ installation folder etc directory
- Mask key store and trust store passwords using Artemis tool: `./artemis mask <plaintextPassword>`
- Change acceptor url to the following: `<acceptor name="artemis">tcp://0.0.0.0:26010?sslEnabled=true;...ord=ENC(masked password);trustStorePath=../etc/broker.ts;trustStorePassword=ENC(masked password);needClientAuth=true;enabledProtocols=TLSv1,TLSv1.1,TLSv1.2;tcpSendBufferSize=1048576;tcpReceiveBufferSize=1048576`

ActiveMQ JMS/TLS

This section and subsections describe how clients can access the RTG MB when the MB is using ActiveMQ Artemis as the JMS messaging service and the JMS messages are secured by TLS. === RTG Available JMS Topics JMS uses Topics to separate and specify subject data to which consumers may subscribe. RTG publishes several JMS Topics that may be of interest to consumers (clients):

- **Nexidia.RTG.Telephony:** Discrete Computer Telephony Interface (CTI) events that occur for each call: CallStart, CallPause (hold), CallResume (off hold), CallEnd.
- **Nexidia.RTG.Notifications:** Discrete Events that occur during a call based on phrases spoken by the agents and the customer participating in the call.
- **Nexidia.RTG.Updates:** Periodically updated array of Enlighten (sentiment) scores for a call.

<https://nem-doshub-mindtouch-us/Nexidia-Analytics-and-Data-Exchange-Framework/NXQC-12.5/Grid-Documentati...>

Updated: Thu, 30 Apr 2026 18:12:29 GMT

- **Nexidia.RTG.Transcript:** Periodically updated array of all words spoken on each leg (Agent or Customer) of a call. Numbers are redacted from the transcript to prevent sending sensitive information such as Credit Card numbers, account information, Social Security Numbers, etc.
- **Nexidia.RTG.Transcript.Finalized:** Periodically updated array of all finalized words spoken on each leg (Agent or Customer) of a call. Numbers are redacted from the transcript to prevent sending sensitive information such as Credit Card numbers, account information, Social Security Numbers, etc.

Python Client

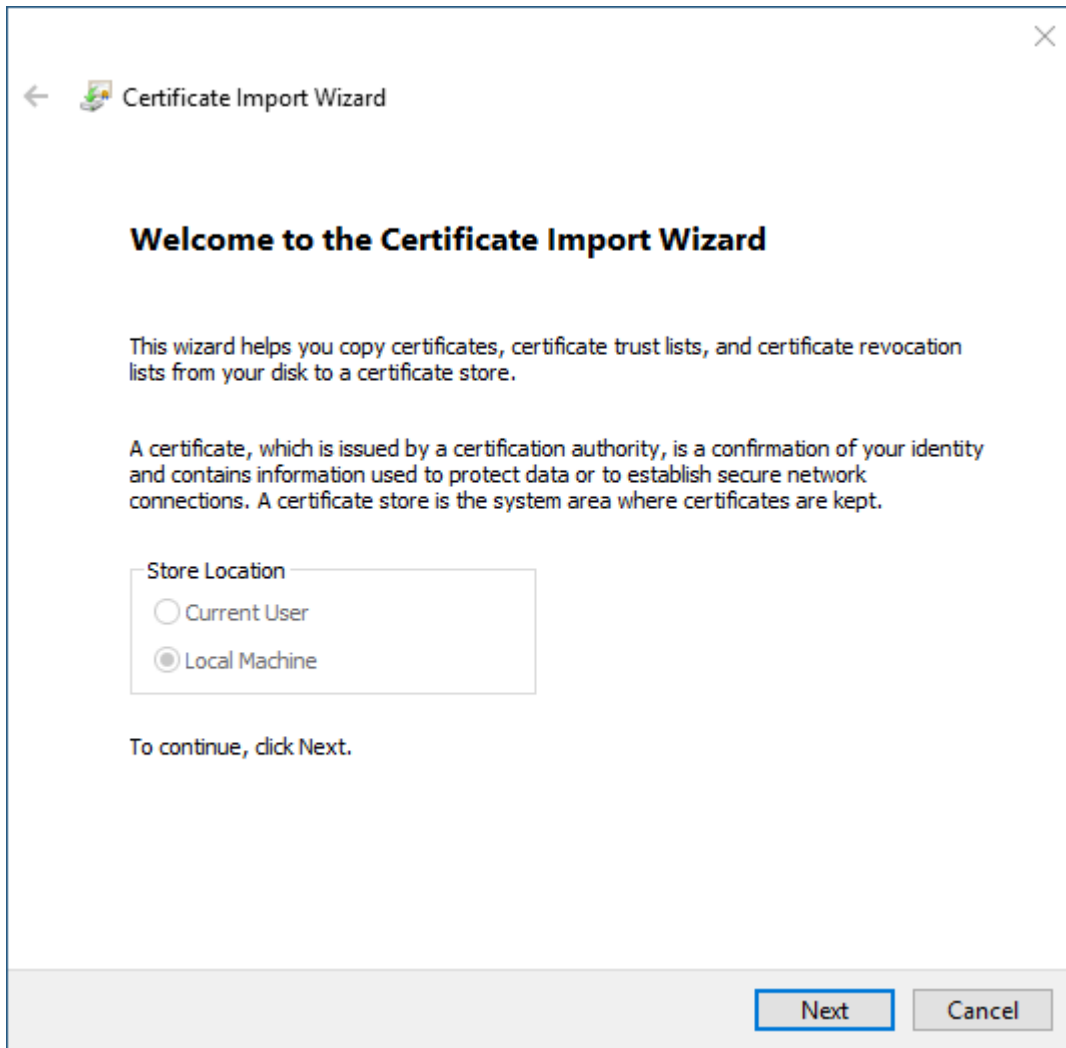
A Python client can access the RTG MB JMS messages using the Stomp Python module for connectivity and the SSL Python module for security. The following minimum versions are recommended. Older versions are not supported:

- Python 3.9.10
- Stomp 8.0.1
- SSL 21.0.0

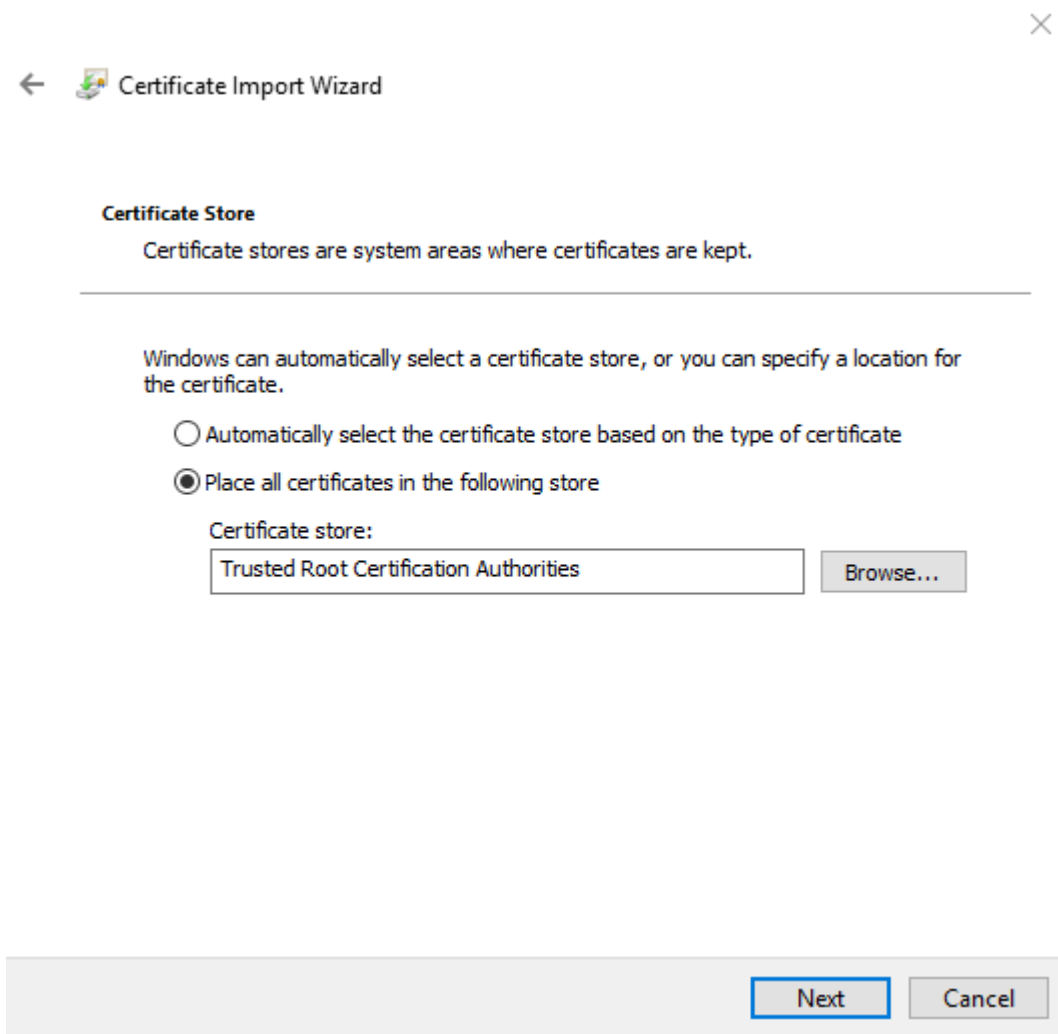
Python Client Trusted Root CA Import

Import the Trusted Root CA Certificate, and any intermediate certificates, to the Trusted Root Authority Certificates folder in the Windows Certificate Store:

1. Open Start → type certlm.msc. The Certificate Manager opens.
2. Expand "Trusted Root Certification Authorities" in the left pane, right-click the Certificate folder; and choose All Tasks → Import. The Certificate Import Wizard opens.



3. Select Next, then Browse to the Trusted Root CA certificate file to import. Select Next.
4. Confirm that "Place all certificates in the following store" is selected, and that the Certificate Store Text Box reads "Trusted Root Certification Authorities."



5. Select Next, then on the next screen select Finish. A pop-up should confirm that "The import was successful."

Python Client Certificate

A Python client will require a client key and certificate for authentication. The key and certificate need only reside on the client machine. The client certificate does not need to be imported to the RTG MB machine.

When the Trusted Certificate is acquired, it will be accompanied by a Certificate Signing Request (CSR). The CSR is required in the generation of the client certificate. The steps below refer to the CSR for example as client.csr.

Use OpenSSL to Generate the Client Key and Certificate

Obtain and unzip OpenSSL from Downloads on the OpenSSL site at <https://www.openssl.org/source/>

Use OpenSSL and the CSR that accompanied the Trusted Certificate in the following steps to generate a client key and certificate for communication with the RTG MB:

1. `openssl genrsa -out client.key 4096`
2. `openssl x509 -req -days 365 -in client.csr -signkey client.key -out client.pem`

3. The CSR (client.csr) is not needed after this step.

https://www.dochub.mindtouch.us/Nexidia_Analytics_and_Data_Exchange_Framework/NXQC_12.5/Grid_Documentati...

Updated: Thu, 30 Apr 2026 18:12:29 GMT

4. Place the client.key and client.pem in a location accessible by the Python client.

5. Use the key and PEM in the set_ssl function in the Python client code. Ex:

```
conn.set_ssl(for_hosts=[(AMQHOST, AMQPORT)], key_file='client.key', cert_file='client.pem',  
ssl_version=ssl.PROTOCOL_TLSv1_2)
```

6. Acquire the user and password used by the RTG MB for authentication. Use them, for example, to set AMQUSER and AMQPASS. Pass these values to the Stomp connection invocation. Ex:

```
conn.connect(AMQUSER, AMQPASS, wait=True)
```

Python Example Code

Use Python Stomp, Python SSL, and the certificate created above to create code to connect to the RTG MB and receive JMS messages secured by TLS.

The example assumes the RTG machine is hosted by Nexidia and is configured to receive TLS traffic on port 443. The example assumes port 443 is forwarded to the RTG MB default port 26010.

For this example it is assumed the Python example code is in a file my.python.jms.tls.example.py.

```
import time  
import sys  
import stomp  
import ssl  
import json  
  
class MsgListener(stomp.ConnectionListener):  
    def on_error(self, headers, message):  
        print('received an error "%s"' % message)  
    def on_message(self, message):  
        print('Received message "%s"' % message)  
  
AMQHOST="myhostedrtg.exampdomain.com"  
AMQPORT=443  
AMQUSER="admin"  
AMQPASS="password"  
TOPICNAME="Nexidia.RTG.Transcript"  
hosts = [(AMQHOST, AMQPORT)]  
conn = stomp.Connection(host_and_ports=hosts)  
conn.set_ssl(for_hosts=[(AMQHOST, AMQPORT)], key_file='client.key', cert_file='client.pem', ssl_  
version=ssl.PROTOCOL_TLSv1_2)  
conn.set_listener('', MsgListener())  
conn.connect(AMQUSER, AMQPASS, wait=True)  
print('Connected to "%s"' % AMQHOST)  
  
conn.subscribe('/topic/'+TOPICNAME, id=1, ack='auto')  
print('Subscribed to "%s"' % TOPICNAME)  
  
time.sleep(300)  
conn.disconnect()
```

Run the Python Example

Run the Python example client from a Windows command prompt

Updated: Thu, 30 Apr 2026 18:12:29 GMT

```
python my.python.jms.tls.example.py
```

The Python example code should connect to the RTG MB, subscribe to the Transcript Topic, and output Transcript JMS messages to the console.

Java Client

A Java client can access the RTG MB JMS messages using Java New Input/Output (NIO), the Apache ActiveMQ package for connectivity, and a Java Truststore and Keystore for security. The following minimum versions are recommended. Older versions are not supported.

- Java 8
- Apache ActiveMQ Client 5.16.1

The provided example uses JSON Simple for JSON parsing. The following JARs are sufficient to build the example:

- activemq-all-5.16.1.jar
- json_simple-1.1.jar

Java Client Certificate

A Java client will require a client key and certificate for authentication. The certificate need only be imported to the Java Truststore and Keystore used by the client. The client certificate does not need to be imported to the RTG MB machine.

Use OpenSSL to Generate the Client Key and Certificate

Obtain and unzip OpenSSL from Downloads on the OpenSSL site at <https://www.openssl.org/source/>

Use OpenSSL to create a client certificate signed by the Trusted Root CA Certificate for communication with the RTG MB. In this example we refer to the Trusted Root CA as my.trusted.root.pfx and its Private Key as my.private.key.

The Trusted Root CA Certificate will be provided by an export from the RTG server, and will likely be in Personal Information Exchange (PFX) format. The PFX should have an icon with a key symbol on it, indicating it includes the Private Key.

Convert the PFX into separate Privacy-Enhanced Mail (PEM) and Private Key files:

1. openssl pkcs12 -in my.trusted.root.pfx -out file.withkey.pem
2. openssl rsa -in file.withkey.pem -out my.private.key

Use OpenSSL to create a client key and Certificate Signing Request (CSR):

3. openssl genrsa -out device_cert_key_filename.key 4096
4. openssl req -new -key device_cert_key_filename.key -out device_cert_csr_filename.csr

Use OpenSSL to create a client PEM signed by the Trusted Root PEM, then convert the client PEM to x.509 Distinguished Encoding Rules (DER) format for import to the client Java Truststore and Keystore.

Updated: Thu, 30 Apr 2026 18:12:29 GMT

1. `openssl x509 -req -in device_cert_csr_filename.csr -CA file.withkey.pem -CAkey my.private.key -CAcreateserial -out my.client.pem -days 500 -sha256`
2. `openssl x509 -outform der -in my.client.pem -out my.client.der`

Java Client Trusted Root CA Import

Use Java Keytool to import the Client DER into the Java Truststore and Keystore that will be used by the Java client. Keytool (keytool.exe) is found in the bin directory of the Java Software Development Kit (SDK).

The Java SDK or Java Runtime Environment (JRE) includes a Trusted Root Certificate Authority list in (SDK root directory)\jre\lib\security\cacerts. It is recommended that a copy of the cacerts be made and used as the client Truststore, leaving the original in place and intact. It is also recommended that the user change the Truststore password from the default "changeit" to something more secure.

This example, refers to the client's copy of cacerts as my.client.ts. Use Keytool to set a new password for my.client.ts:

1. `keytool -storepasswd -keystore my.client.ts`

When prompted for the existing password, enter the default "changeit". When prompted for a new password enter something more secure. The new password will be required when importing new certificates (below) to my.client.ts.

Use Keytool to import the DER created above to the client Truststore and Keystore. If no Keystore exists the following steps will create a new client Keystore. The Alias set below should be changed to something readily recognized as the client's signed certificate:

2. `keytool -import -alias my.client.certificate -keystore my.client.client.ts -file my.client.der`
3. `keytool -import -alias my.client.certificate -keystore my.client.ks -file my.client.der`

Java Example Code

Use Java, NIO, Apache ActiveMQ, JSON Simple, and the Truststore and Keystore created above to create code to connect to the RTG MB and receive JMS messages secured by TLS.

The example assumes the RTG machine is hosted by Nexidia and is configured to receive TLS traffic on port 443. The example assumes port 443 is forwarded to the RTG MB default port 26010.

The example client accepts the full URI of the RTG MB as its only argument. Ex:

`nio+ssl://<RTG host machine address>:443`

Java Example Client Main Class

The main class initializes the TranscriptConsumer on its own Thread.

```

package com.nexidia.jms.tls.example;

import java.nio.file.Paths;

public final class JavaJMSExampleClient {

    public static void main(String[] args) {
        final String brokerUri = args[0];

        System.out.println("ScanProxy starting");
        System.out.println("Current run dir: " + Paths.get("").toAbsolutePath());

        try {
            final Thread transcriptConsumer = new Thread(new TranscriptConsumer(brokerUri));
            transcriptConsumer.setDaemon(false);
            transcriptConsumer.start();

        } catch (final Exception x) {
            System.out.println("ScanProxy ERROR: " + x);
        }
    }
}

```

Java Example Client Transcript Consumer

The Transcript Consumer connects to the RTG MB using the URI passed as the only argument to the Main class. The Transcript Consumer subscribes to the Transcript Topic and outputs Transcript JMS messages to the console.

```

package com.nexidia.jms.tls.example;

import java.util.Enumeraation;

import javax.jms.Connection;
import javax.jms.Destination;
import javax.jms.ExceptionListener;
import javax.jms.JMSException;
import javax.jms.Message;
import javax.jms.MessageConsumer;
import javax.jms.MessageListener;
import javax.jms.Session;
import javax.jms.TextMessage;

import org.apache.activemq.ActiveMQConnectionFactory;
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
import org.json.simple.parser.ParseException;

public class TranscriptConsumer implements Runnable, MessageListener, ExceptionListener {

    private final String connectionUri;
    private final JSONParser parser;

    public TranscriptConsumer(String connectionUri) {
        this.connectionUri = connectionUri;
        parser = new JSONParser();
    }

    @Override
    public void run() {
        System.out.println("TranscriptConsumer starting");

        try {

            // Create a ConnectionFactory
            final ActiveMQConnectionFactory connectionFactory = new
ActiveMQConnectionFactory(connectionUri);

            // Create a Connection
            final Connection connection = connectionFactory.createConnection();
            connection.start();

            connection.setExceptionListener(this);

            // Create a Session
            final Session session = connection.createSession(false, Session.AUTO_ACKNOWLEDGE);

            // Create the destination (Topic or Queue)
            final Destination destination = session.createTopic("Nexidia.RTG.Transcript");

            // Create a MessageConsumer from the Session to the Topic or Queue
            final MessageConsumer consumer = session.createConsumer(destination);
            consumer.setMessageListener(this);

            System.out.println("TranscriptConsumer ready");

            // Wait for a message
            try {

                while (true) {
                    Thread.sleep(10000);

```

```

        }

    } finally {
        consumer.close();
        session.close();
        connection.close();
    }

} catch (final Exception x) {
    System.out.println("Caught: " + x);
}

}

@Override
public void onMessage(Message message) {
    try {
        if (message instanceof TextMessage) {
            final TextMessage textMessage = (TextMessage) message;
            final String text = textMessage.getText();

            final Enumeration<?> srcProperties = message.getPropertyNames();

            while (srcProperties.hasMoreElements()) {
                String propertyName = (String)srcProperties.nextElement();
                System.out.println(propertyName + ":" + (String)message.
getObjectProperty(propertyName));
            }

            final JSONObject json = (JSONObject) parser.parse(text);
            System.out.println("Transcript Received: " + json.toJSONString());
        }

    } catch (final JMSEException x) {
        System.out.println("Message error: " + x);

    } catch (final ParseException x) {
        System.out.println("JSON parser error: " + x);
    }

}

@Override
public void onException(JMSEException arg0) {
    System.out.println("JMS Exception occurred. Shutting down client.");
}

}

```

Java Example Client Command File

For this example, it is assumed the source code uses the package structure shown in the example code, that the client.ks and client.ts are in the runtime directory, and that all JAR dependencies are in a "lib" directory:

Create and edit the Java Example Client Command File. In this example, it is assumed the command file is named my.java.jms.tls.example.cmd

```
@pushd %~p0
@echo off
@title java-jms-tls-example

"%JAVA_HOME\bin\java.exe" ^
-Djavax.net.ssl.keyStore=./my.client.ks ^
-Djavax.net.ssl.keyStorePassword=mykeystorepassword ^
-Djavax.net.ssl.trustStore=./my.client.ts ^
-Djavax.net.ssl.trustStorePassword=mytruststorepassword ^
-classpath ^
. ; ../lib/* com.nexidia.jms.tls.example.JavaJMSEExampleClient %1
@popd
```

Place the `my.java.jms.tls.example.cmd` at the top of the directory structure for the compiled Java code, i.e. in the same directory that contains the top-level "com" directory.

Run the example command file:

```
my.java.jms.tls.example.cmd nio+ssl://<RTG MB host machine address>:443
```

The example code should connect to the RTG MB, subscribe to the Transcript topic, and output Transcript messages to the console.